

Reporte de Analisis y Hardening de Seguridad

API PoliCard

Controles evaluados e implementados:

API Key | JWT | Analisis BOLA/IDOR y Roles | Rate Limiter

Fecha: 10/07/2026

Proyecto: PoliCard - Plataforma de tarjetas de credito para jovenes

Integrantes:

María de los Ángeles Palma Torres

Marlon Raúl Mendieta Valadez

María Guadalupe Jiménez Ruiz

Jorge Armando López Morales

1. Resumen Ejecutivo

PoliCard es una plataforma web (FastAPI + PostgreSQL) que conecta bancos y clientes jovenes para la contratacion de tarjetas de credito. El backend operaba unicamente con autenticacion por cookie de sesion (server-rendered) y no exponia una API JSON real ni mecanismos de proteccion contra fuerza bruta o abuso de recursos.

Se realizo un analisis de seguridad enfocado en el OWASP API Security Top 10, con especial atencion a Broken Object Level Authorization (BOLA/IDOR), Broken Authentication y Unrestricted Resource Consumption. Como resultado se identifico y corrigio una vulnerabilidad BOLA/IDOR real y explotable, y se implementaron los 4 controles solicitados:

- API Key: protege la nueva API JSON /api/v1 (consumida por la app movil), identificando a la aplicacion cliente autorizada.
- JWT: autentica al usuario final (cliente o banco) en /api/v1, con expiracion de 60 minutos.
- Analisis BOLA/IDOR + Roles: se audito cada endpoint con IDs en la ruta; se corrigio la vulnerabilidad encontrada y se centralizo la verificacion de roles.
- Rate Limiter: se limitaron los endpoints de login/registro y la API movil para mitigar fuerza bruta y abuso.

Todos los cambios fueron verificados de forma automatizada (ver Seccion 5, Evidencia de Pruebas) contra una instancia real de la aplicacion.

2. Metodologia

El analisis siguio un enfoque basado en el OWASP API Security Top 10 (2023), revisando cada router de la aplicacion (publicas, auth, admin, banco, cliente) linea por linea:

- API1 - Broken Object Level Authorization: para cada endpoint con un {id} en la ruta, se verifico si la consulta a base de datos restringe el recurso al usuario/rol autenticado o si acepta cualquier id.
- API2 - Broken Authentication: se reviso el mecanismo de sesion (secret hardcodeado) y la ausencia de un esquema de autenticacion para clientes no-navegador (apps moviles).
- API4 - Unrestricted Resource Consumption: se identificaron endpoints publicos sin limite de tasa (login, registro), vulnerables a fuerza bruta y spam.
- API5 - Broken Function Level Authorization: se verifico que los chequeos de rol (admin/banco/cliente) esten presentes y sean consistentes en todas las rutas.

3. Hallazgo Critico: BOLA/IDOR en app/routers/banco.py

Severidad: ALTA

Los endpoints aprobar_solicitud_cliente, rechazar_solicitud_cliente y cancelar_tarjeta_cliente validaban que el usuario autenticado tuviera rol "banco", pero consultaban la solicitud unicamente por su id, sin verificar que perteneciera a una tarjeta del banco autenticado:

Codigo vulnerable (antes)

```
sol = db.query(SolicitudTarjeta).filter(SolicitudTarjeta.id == id).first()
```

```
sol.estado = "aprobada" # cualquier banco autenticado puede modificar
                        # solicitudes de OTRO banco iterando ids
```

Impacto: cualquier representante de banco autenticado (ej. Santander) podia aprobar, rechazar o cancelar solicitudes de clientes que pertenecen a tarjetas de OTRO banco (ej. BBVA), simplemente adivinando/iterando el parametro {id} en la URL. Esto viola la confidencialidad e integridad de datos de negocio entre bancos competidores.

Codigo corregido (despues)

```
tarjeta_ids = [t.id for t in banco.tarjetas]
sol = db.query(SolicitudTarjeta).filter(
    SolicitudTarjeta.id == id,
    SolicitudTarjeta.tarjeta_id.in_(tarjeta_ids)
).first()
if sol is None:
    raise Exception("No encontrada") # 404 / redirect, sin modificar nada
```

Este es el mismo patron que ya usaban correctamente `editar_tarjeta_post` y `eliminar_tarjeta` en el mismo archivo (filtrando por `Tarjeta.banco_id == banco.id`) -- se replico ese patron seguro. El mismo fix se aplico tambien en el endpoint equivalente de la nueva API movil (POST `/api/v1/banco/solicitud-cliente/{id}/aprobar`).

4. Controles de Seguridad Implementados

4.1 API Key

Header requerido: X-API-Key. Protege todos los endpoints de `/api/v1` (app movil), identificando que la peticion proviene de la aplicacion oficial y no de un cliente arbitrario.

```
# app/security/api_key.py
def verify_api_key(key: str | None = Depends(_api_key_header)) -> None:
    if key is None or key != API_KEY:
        raise HTTPException(status_code=401, detail="API key invalida o ausente")

# app/routers/api.py
router = APIRouter(prefix="/api/v1", dependencies=[Depends(verify_api_key)])
```

4.2 JWT (JSON Web Token)

Algoritmo HS256, expiracion de 60 minutos. Se emite en POST `/api/v1/auth/login` tras validar credenciales, y se exige via header `Authorization: Bearer <token>` en los endpoints de cliente y banco de la API movil.

```
# app/security/jwt_auth.py
def create_access_token(usuario: Usuario) -> str:
    payload = {"sub": str(usuario.id), "tipo": usuario.tipo,
               "iat": now, "exp": now + timedelta(minutes=60)}
    return jwt.encode(payload, JWT_SECRET_KEY, algorithm="HS256")

def require_role_jwt(*roles: str):
    def dependency(usuario: Usuario = Depends(get_current_user_jwt)):
        if usuario.tipo not in roles:
            raise HTTPException(status_code=403, detail="No tienes permisos")
        return usuario
```

```
return dependency
```

4.3 Analisis BOLA/IDOR + Roles

Ademas del fix critico (Seccion 3), se centralizo la verificacion de roles en rutas web con sesion (app/security/roles.py: check_role), reemplazando el chequeo inline duplicado en las 10 rutas de app/routers/admin.py -- reduce el riesgo de que una ruta nueva olvide el control de acceso. En la API movil, require_role_jwt(*roles) cumple el mismo proposito. El id del usuario en los endpoints de cliente (/api/v1/cliente/*) se obtiene siempre del claim 'sub' del JWT, nunca de un parametro enviado por el cliente -- eliminando la superficie de ataque BOLA en esas rutas por diseno.

4.4 Rate Limiter

Implementado con slowapi (limite por IP de origen). Se aplico a los endpoints de mayor riesgo de fuerza bruta / abuso:

Endpoint	Limite	Motivo
POST /login	5/min	Fuerza bruta de credenciales
POST /registro_banco	5/min	Spam de registros
POST /registro	5/min	Spam de registros
POST /api/v1/auth/login	5/min	Fuerza bruta (app movil)
GET /api/v1/tarjetas	30/min	Uso normal de catalogo, limite generoso
POST /api/v1/cliente/tarjeta/{id}/solicitar	10/min	Evitar spam de solicitudes

5. Evidencia de Pruebas

Se ejecuto un script de verificacion automatizada (TestClient de FastAPI + SQLite) que levanta la aplicacion completa y ejerce cada control contra datos reales de la base de datos. Salida real capturada:

5.1 Rate limiter - fuerza bruta en /login (limite 5/min)

```
[200] Intento login #1
[200] Intento login #2
[200] Intento login #3
[200] Intento login #4
[200] Intento login #5
[429] Intento login #6 <-- bloqueado por rate limiter
```

5.2 API Key en /api/v1/tarjetas

```
[401] GET /api/v1/tarjetas SIN X-API-Key
[401] GET /api/v1/tarjetas con X-API-Key INCORRECTA
[200] GET /api/v1/tarjetas con X-API-Key correcta -- 12 tarjetas
```

5.3 JWT en /api/v1

```
[401] POST /api/v1/auth/login credenciales invalidas
[200] POST /api/v1/auth/login credenciales correctas
[401] GET /api/v1/cliente/me SIN Authorization
[401] GET /api/v1/cliente/me con JWT invalido
[200] GET /api/v1/cliente/me con JWT valido -- {'id': 8, 'nombre': 'Cliente Test',
      'email': 'cliente_test@poliscard.com', 'tipo': 'cliente'}
[403] GET /api/v1/cliente/me con JWT de rol 'banco' (rol incorrecto)
```

5.4 BOLA/IDOR corregido - API movil (/api/v1/banco/solicitud-cliente/{id}/aprobar)

```
[404] Banco Santander intenta aprobar solicitud #1 (pertenece a BBVA) -- BLOQUEADO
[200] Banco BBVA (dueno real) aprueba solicitud #1 -- PERMITIDO
```

5.5 BOLA/IDOR corregido - ruta web (/banco/solicitud-cliente/{id}/rechazar)

```
[302] Banco Santander (web) intenta rechazar solicitud #2 de BBVA
[verificacion] Estado tras intento cruzado: 'pendiente' (NO se modifico) -- CORRECTO
[302] Banco BBVA (dueno real, web) rechaza solicitud #2
[verificacion] Estado final: 'rechazada' -- CORRECTO
```

5.6 Roles en ruta admin (/admin/dashboard)

```
[302] GET /admin/dashboard sin sesion -- redirige a /login
[302] GET /admin/dashboard con sesion tipo 'cliente' -- redirige (sin permiso)
[200] GET /admin/dashboard con sesion tipo 'admin' -- permitido
```

5.7 Control de versiones - commit de las implementaciones

Los cambios descritos en este reporte fueron confirmados (commit) en el repositorio del proyecto:

```
Mensaje: "Implementaciones de seguridad"
Detalle: "Entregable PI - SI"
Autor:   Maria-Palma0731
Hash:    4efa00f
```

Archivos: 14 changed files (+361 / -34)

File Edit View Repository Branch Help

Current repository

policardd

Current branch

main

Fetch origin

Last fetched just now

Changes 22

History

Implementaciones de seguridad

Entregable PI - SI

Maria-Palma0731

4efa00f

14 changed files

app/main.py

app\models\schemas.py

app\routers\admin.py

app\routers\api.py

app\routers\auth.py

app\routers\banco.py

app\routers\cliente.py

app\security\api_key.py

app\security\config.py

app\security\jwt_auth.py

app\security\rate_limit.py

app\security\roles.py

docker-compose.yml

requirements.txt

app/main.py

@@ -1,8 +1,13 @@

1 1 from fastapi import FastAPI

2 2 from fastapi.staticfiles import StaticFiles

3 3 from starlette.middleware.sessions import SessionMiddleware

are

4 + from slowapi import _rate_limit_exceeded_handler

5 + from slowapi.errors import RateLimitExceeded

6 + from slowapi.middleware import SlowAPIMiddleware

7 from app.data.db import engine, Base

4 7

5 - from app.routers import publicas, auth, admin, banco, cliente

8 + from app.routers import publicas, auth, admin, banco, cliente, api

9 + from app.security.config import SESSION_SECRET_KEY

10 + from app.security.rate_limit import limiter

6 11

7 12 # Importar modelos para que SQLAlchemy los registre

8 13 from app.data import usuario, banco as bancoModel, tarjeta, solicitud

@@ -18,8 +23,13 @@ app = FastAPI(

18 23 version="1.0.0"

19 24)

20 25

26 + # Rate limiting (slowapi)

27 + app.state.limiter = limiter

28 + app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

6. Inventario de Endpoints y Necesidades de Seguridad

Clasificación de todos los endpoints del backend. Estado: [OK] ya seguro sin cambios, [FIX] vulnerabilidad corregida, [NUEVO] control agregado, [REC] recomendación para trabajo futuro (fuera del alcance de esta entrega).

6.1 Rutas publicas (app/routers/publicas.py)

Endpoint	Auth	Control de seguridad	Estado
GET /	Publico	Ninguna (sin datos sensibles)	OK
GET /tarjetas	Publico	Ninguna (catalogo publico)	OK
GET /buscar	Publico	Ninguna (filtros validados)	OK
GET /educacion	Publico	Ninguna (contenido estatico)	OK
GET /calculadora	Publico	Ninguna (contenido estatico)	OK

6.2 Autenticacion (app/routers/auth.py)

Endpoint	Auth	Control de seguridad	Estado
GET /login	Publico	Ninguna	OK
POST /login	Publico	Rate limiter (fuerza bruta)	NUEVO
GET /perfil	Sesion	Sesion valida	OK
GET /registro_banco	Publico	Ninguna	OK
POST /registro_banco	Publico	Rate limiter (spam)	NUEVO
GET /logout	Sesion	Ninguna	OK
GET /dashboard	Sesion	Sesion valida	OK

6.3 Admin (app/routers/admin.py)

Endpoint	Auth	Control de seguridad	Estado
GET /admin(/dashboard)	Sesion	Rol admin (check_role)	NUEVO
GET /admin/solicitudes	Sesion	Rol admin (check_role)	NUEVO
POST /admin/solicitud/{id}/aprobar	Sesion	Rol admin; alcance global por diseno	OK
POST /admin/solicitud/{id}/rechazar	Sesion	Rol admin; alcance global por diseno	OK
GET /admin/bancos	Sesion	Rol admin (check_role)	NUEVO
GET /admin/tarjetas	Sesion	Rol admin (check_role)	NUEVO
POST /admin/tarjeta/{id}/aprobar	Sesion	Rol admin; alcance global por diseno	OK
GET /admin/tarjeta/{id}/tendencia	Sesion	Rol admin (JSON, ya protegido)	OK
GET /admin/usuarios	Sesion	Rol admin (check_role)	NUEVO
POST /admin/usuarios/crear-admin	Sesion	Rol admin (check_role)	NUEVO
POST /admin/usuario/{id}/toggle	Sesion	Rol admin (check_role)	NUEVO

6.4 Banco (app/routers/banco.py)

Endpoint	Auth	Control de seguridad	Estado
GET /banco/dashboard	Sesion	Rol banco (scoped)	OK
POST /banco/perfil/editar	Sesion	Rol banco (scoped)	OK
GET /banco/tarjetas	Sesion	Rol banco (scoped)	OK

Endpoint	Auth	Control de seguridad	Estado
GET-POST /banco/tarjeta/nueva	Sesion	Rol banco + banco.aprobado	OK
GET-POST /banco/tarjeta/{id}/editar	Sesion	Filtra por banco_id (BOLA-safe)	OK
POST /banco/tarjeta/{id}/eliminar	Sesion	Filtra por banco_id (BOLA-safe)	OK
GET /banco/solicitudes-clientes	Sesion	Filtra por tarjeta_ids (BOLA-safe)	OK
POST /banco/solicitud-cliente/{id}/aprobar	Sesion	Filtro BOLA agregado	FIX
POST /banco/solicitud-cliente/{id}/rechazar	Sesion	Filtro BOLA agregado	FIX
GET /banco/clientes	Sesion	Filtra por tarjeta_ids (BOLA-safe)	OK
POST /banco/clientes/{id}/cancelar	Sesion	Filtro BOLA agregado	FIX

6.5 Cliente (app/routers/cliente.py)

Endpoint	Auth	Control de seguridad	Estado
GET /registro	Publico	Ninguna	OK
POST /registro	Publico	Rate limiter (spam)	NUEVO
POST /tarjeta/{id}/solicitar	Sesion	Filtra por usuario_id (BOLA-safe)	OK

6.6 API movil nueva (app/routers/api.py)

Endpoint	Auth	Control de seguridad	Estado
POST /api/v1/auth/login	API Key	Rate limiter 5/min	NUEVO
GET /api/v1/tarjetas	API Key	Rate limiter 30/min	NUEVO
GET /api/v1/cliente/me	API Key + JWT	Rol cliente; id desde JWT	NUEVO
GET /api/v1/cliente/solicitudes	API Key + JWT	Rol cliente; scoped a usuario_id	NUEVO
POST /api/v1/cliente/tarjeta/{id}/solicitar	API Key + JWT	Rol cliente; rate limiter 10/min	NUEVO
GET /api/v1/banco/solicitudes-clientes	API Key + JWT	Rol banco; scoped a tarjeta_ids	NUEVO
POST /api/v1/banco/solicitud-cliente/{id}/aprobar	API Key + JWT	Rol banco; filtro BOLA	NUEVO

7. Recomendaciones Pendientes (fuera de alcance de esta entrega)

- HTTPS/TLS obligatorio en produccion (hoy el contenedor expone HTTP plano en el puerto 10000).
- Rotar JWT_SECRET_KEY, API_KEY y SESSION_SECRET_KEY a valores unicos y secretos en el entorno de produccion (actualmente hay un fallback de desarrollo que emite una advertencia si no se configuran).
- Refresh tokens y revocacion de JWT (hoy el token no se puede invalidar antes de su expiracion de 60 min).
- Logging de auditoria para intentos fallidos de autenticacion y accesos denegados por rol (hoy solo se registran acciones administrativas en AdminLog).
- Mover el rate limiter a un backend compartido (Redis) si se despliega en multiples instancias -- la implementacion actual usa memoria local por proceso.
- Agregar CSRF tokens a los formularios web (login, registro) ya que usan cookies de sesion.
- Endpoint /admin/tarjeta/{id}/tendencia: aunque esta correctamente protegido por sesion+rol, no se migro a la API JSON/JWT porque no es consumido por la app movil; si en el futuro se expone fuera del panel admin, debe recibir el mismo tratamiento.

8. Conclusion

Se corrigio una vulnerabilidad BOLA/IDOR de severidad alta que permitia a un banco autenticado manipular solicitudes de clientes de otros bancos. Se implementaron los 4 controles de seguridad solicitados (API Key, JWT, roles/BOLA, Rate Limiter), aplicandolos de forma concreta tanto en las rutas web existentes como en una nueva API JSON /api/v1 pensada para la app movil. Todos los controles fueron verificados con pruebas automatizadas que confirman el comportamiento esperado (bloqueo cuando corresponde, acceso permitido cuando corresponde). El inventario completo de endpoints (Seccion 6) documenta el estado de seguridad de cada ruta del sistema, incluyendo las que no requirieron cambios por ya ser publicas o estar correctamente protegidas.